Installation and demo

Checkpoint System saves a player's position and rotation at a checkpoint and returns the player there after a respawn request. Each manager controls one player in one scene. The default store writes progress to a local JSON file.

Requirements

Developed for Unity 2022.3 LTS. Tested with Unity 2022.3.62f3 on Windows using the Built-in Render Pipeline. The runtime uses 3D physics and has no external package dependencies. The demo materials require conversion for URP or HDRP.

Open the demo

1. Import CheckpointSystem-1.0.0.unitypackage into your project.
2. Select Tools > Checkpoint System > Open Demo Scene.
3. Press Play. Move the rover into the Relay checkpoint zone.
4. Press F to fall. The rover should return to the saved spawn point.
5. Stop Play mode and start it again to check that the save is restored.

Control	Action
WASD / arrow keys	Move
Space	Jump
R	Request a respawn
F	Fall into the recovery zone
L	Load the saved checkpoint
X	Clear this scene and slot
C	Switch camera view
Tab	Show or hide the interface

Run guided tour moves the rover through all three checkpoints using the scene colliders. It clears the demo slot before starting. Stop guided tour returns manual control.

The scene is Scenes/Checkpoint_Showcase.unity. It uses scene key checkpoint-showcase-v1 and save slot showcase-player. Keyboard movement uses the legacy input manager, with an IMGUI fallback. A project configured exclusively for the new Input System has not been independently tested.

Documentation

Manual.md, Manual.txt and Checkpoint-System-Manual.pdf contain the same instructions. The PDF is also available through Tools > Checkpoint System > Open English Manual.

Scene setup

1. Add the player target

Add RespawnTarget to the player root that owns the CharacterController or Rigidbody. Keep the root scale at 1. Child colliders can detect this component through their parent hierarchy. The Demo Explorer prefab provides a working CharacterController example.

2. Assign a manager

Add the Checkpoint Manager prefab, then assign the player's RespawnTarget to Player. Keep the manager and its checkpoints in the same scene. Use one manager per player and different save slots for separate players.

Initial Spawn is optional. If assigned, its world position and rotation become the starting pose. Otherwise, the player's pose is captured during initialization. Moving Initial Spawn later does not update that captured pose.

3. Place checkpoints

Place the Checkpoint Beacon prefab where the player should save progress. Set its display name and order. Each checkpoint needs a unique ID. After duplicating an instance, use Generate New Checkpoint ID in the Checkpoint component's context menu.

Position the Spawn Point child at the required player root location, with the desired facing direction. Keep it above solid ground and clear of hazards and obstacles. For the demo rover, the root sits approximately 1.12 units above the deck.

4. Check trigger settings

Use a trigger BoxCollider and a kinematic Rigidbody with Use Gravity disabled on the checkpoint. The supplied prefab already includes these components. Player Layers must include the incoming collider's layer, and the Physics collision matrix must allow the two layers to interact.

5. Add recovery zones

Use the Respawn Zone prefab for pits or hazards. Add FallRespawn to the player for recovery below a minimum world Y coordinate. A health system can call CheckpointManager.RequestRespawn() directly.

6. Validate

Select Tools > Checkpoint System > Validate Open Scenes. Resolve missing player assignments, duplicate IDs and trigger or Rigidbody warnings reported in the Console. Save the scene after setup.

The system records a world-space pose. It does not check for safe ground or follow a moving platform. Reposition spawn points and reset or migrate saved progress when changing the level layout.

Component settings

CheckpointManager

Setting	Behavior
Player	RespawnTarget controlled by this manager.
Initial Spawn	Optional transform used to capture the starting pose.
Scene Key	Stable level identifier. If empty, uses the scene path or name. Set an explicit key before shipping.
Save Slot	Local profile or run identifier. Changing it does not clear memory or load another profile.
Persist To Disk	Saves after checkpoint activation. Explicit save, load and reset calls still access storage when this is off.
Restore On Start	Loads progress at startup when Persist To Disk is also enabled.
Require Increasing Order	Accepts only a strictly higher Order after the first checkpoint.
Respawn Delay	Delay in unscaled seconds. Default: 0.65.

Re-entering the current checkpoint ID does not activate it again or save again. With Require Increasing Order disabled, a different checkpoint can replace the current one even if its order is lower.

Checkpoint

Checkpoint ID identifies the saved point. Display Name is the label used by the demo. Order controls forward-only progression when enabled on the manager. Spawn Point supplies the saved pose; without one, the checkpoint transform is used. Player Layers filters colliders. On Activated runs when the manager accepts the checkpoint.

RespawnTarget

Suspend During Respawn lists movement or camera behaviours to disable during the delay. Their previous enabled states are restored afterward. Do not add RespawnTarget or CheckpointManager to this list. On Teleported runs after the root has moved.

Editor tools

Open Demo Scene opens the supplied showcase. Open English Manual opens the PDF. Validate Open Scenes reports setup issues.

Respawn integration

Call from a death or damage handler

```
using CheckpointSystem;
using UnityEngine;

public sealed class PlayerRecovery : MonoBehaviour
{
    [SerializeField] private CheckpointManager checkpoints;

    public void OnDeath()
    {
        if (checkpoints != null)
            checkpoints.RequestRespawn();
    }
}
```

Assign the manager in the Inspector. For a UnityEvent, bind CheckpointManager.RespawnPlayer instead. RequestRespawn returns false when a request is already running or the player or manager is unavailable.

What happens during a respawn

The target suspends the configured behaviours and the manager invokes On Respawn Started. After the unscaled delay, it teleports the player to the current pose, increments RespawnCount, restores the behaviours and invokes On Respawn Finished. Before the first checkpoint, the current pose is the initial spawn.

Checkpoint activation is blocked during the delay. Disabling the manager cancels the pending respawn and restores suspended behaviours. A zero time scale does not stop the delay. Rigidbody gravity is not suspended automatically.

Movement state

CharacterController is temporarily disabled for teleporting and then returned to its previous enabled state. Rigidbody position and rotation are updated; linear and angular velocity are cleared for a non-kinematic body.

Custom movement code may keep its own velocity, grounded flag or animation state. Reset those values through RespawnTarget.OnTeleported. Use the same event to snap a camera or reposition a NavMeshAgent through its supported API.

Feedback

Use On Checkpoint Activated for save feedback, On Respawn Started for a fade, On Respawn Finished for health or interface updates, and On Progress Reset for a new run. Check LastStorageMessage or subscribe to StorageMessage to display storage errors.

Saving and loading

Saved data

A version 1 record stores sceneKey, checkpointId, displayName, order, position and rotation. It contains the latest checkpoint, not a list of visited checkpoints, inventory, health or other game state. Loading restores the captured pose without requiring the original checkpoint object to exist.

File location

The default JsonCheckpointStore uses Application.persistentDataPath/CheckpointSystem. Each filename is a SHA-256 hash of the save slot and scene key. Separate scene/slot pairs have separate progress. Hashing prevents these names from becoming file paths; it does not encrypt the data.

Save and restore

Persist To Disk saves on accepted activation. If writing fails, the checkpoint remains active in memory. SaveCurrent() explicitly writes the current checkpoint and returns false if no checkpoint has been activated.

LoadSavedCheckpoint() reads the current scene and slot, updates memory and teleports the player. Pass false to update memory without moving the player. Loading does not change scenes and is rejected during a respawn. Automatic startup loading requires both Persist To Disk and Restore On Start.

Reset

ResetProgress() deletes progress for this scene and slot, cancels a pending respawn, clears the counter and returns to the initial pose. Pass false to keep the player in place. If deleting the save fails, the method returns false and preserves in-memory progress.

Validation and backup

The store rejects invalid JSON, unsupported versions, a different scene key, non-finite coordinates, invalid rotation values and files larger than 64 KB. Writes use a temporary file. An existing valid primary is copied to a backup before replacement. Loading tries that backup if the primary is missing or invalid.

The first save has no previous backup. An interrupted first write may lose that save. The file store is local storage and does not provide cloud sync, encryption or tamper protection.

Custom storage

Implement ICheckpointStore and call SetStore before the first save or load. A call before the manager's Start is needed when automatic restore is enabled. The interface is synchronous and runs on the main thread. Use your own queue or cache adapter for asynchronous services.

API reference

Runtime types use the CheckpointSystem namespace. The assembly name is CheckpointSystem.Runtime.

CheckpointManager

Method	Result
Initialize(RespawnTarget target)	Binds one player and captures the initial pose once. Returns success.
TryActivate(Checkpoint checkpoint, RespawnTarget target)	Returns whether the checkpoint was accepted.
RequestRespawn()	Returns whether a respawn request started.
SaveCurrent()	Returns whether the current checkpoint was saved.
LoadSavedCheckpoint(bool teleport = true)	Returns whether valid progress was loaded.
ResetProgress(bool teleport = true)	Returns whether this scene/slot was cleared.
SetStore(ICheckpointStore customStore)	Assigns a non-null storage implementation.

Read state through Player, Current, HasCheckpoint, IsRespawning, RespawnCount, LastStorageMessage and SceneKey. SaveSlot, PersistToDisk, RestoreOnStart, RequireIncreasingOrder and RespawnDelay can be changed from code.

CheckpointChanged passes a CheckpointSnapshot when progress changes. StorageMessage passes status text. Inspector event properties are OnCheckpointActivated, OnRespawnStarted, OnRespawnFinished and OnProgressReset.

The void adapters RespawnPlayer(), SaveProgress(), LoadProgress() and ClearProgress() are available for Inspector UnityEvent bindings. Use the boolean methods when code needs to handle failure.

RespawnTarget

Teleport(Vector3 position, Quaternion rotation) moves the root and invokes OnTeleported. Manager identifies the bound manager. Calling Teleport directly does not update checkpoint progress or the respawn counter.

ICheckpointStore

```
bool TrySave(string slot, CheckpointSnapshot snapshot,
             out string error);
bool TryLoad(string slot, string sceneKey,
             out CheckpointSnapshot snapshot, out string error);
bool TryDelete(string slot, string sceneKey, out string error);
```

Return true on success and supply a useful error message on failure. The manager validates a loaded snapshot and checks its scene key before accepting it.

Troubleshooting and package contents

The checkpoint does not activate

Check the manager's Player assignment, unique checkpoint IDs, enabled components, trigger collider, kinematic Rigidbody, Player Layers and the Physics collision matrix. Keep the manager and checkpoint in the same scene. The current checkpoint ID and lower orders in forward-only mode are intentionally ignored.

The player falls again immediately after respawning

Move Spawn Point above solid ground and outside any recovery trigger. Check the player root height and collider size. Clear any custom movement velocity through OnTeleported. Saved positions do not automatically follow later edits to a spawn point; reset the old save when testing changes.

Progress is not restored

Check Scene Key, Save Slot and LastStorageMessage. Automatic restore requires both persistence flags. The Editor and a built player can use different persistentDataPath locations, depending on project settings. Changing a scene path also changes the default key unless an explicit key is assigned.

The demo materials are pink

The included scene uses the Built-in Render Pipeline. Convert the materials and replace or adapt IndustrialSurface.shader when using URP or HDRP.

Package layout

Folder	Contents
Scripts/Runtime	Checkpoint, respawn and local storage components.
Scripts/Editor	Scene validation and menu commands.
Scripts/Scene	Player movement, terminal indicators and scene interface.
Scenes	Configured industrial checkpoint scene.
Prefabs	Checkpoint Beacon, Checkpoint Manager, Demo Explorer and Respawn Zone.
Art	Eight materials, two textures and one shader.
Documentation	This manual in MD, TXT and PDF; dependency notices.

Tested scope

The Windows scene was checked in Unity 2022.3.62f3 for trigger activation, delayed fall recovery and JSON restore. The supplied publication screenshots were captured from that running player.

Unity 6, other operating systems, URP/HDRP visuals and new-Input-System-only controls were not independently tested. Networking, 2D physics, scene transitions, mobile controls and a complete game-save system are outside this package's scope.